



AI 서비스를 위한 SW 기초 Full-stack Engineering

- 스마트 데이터 이해 및 활용

- Day04_종합실습_평가용

- 2026.07

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

PostgreSQL 프로시저와 함수

기초 문법에서 테이블 연동·트랜잭션·성능·보안까지

실습 스키마 proc_lab | PostgreSQL 14+

수업이 끝나면 ‘선택-구현-검증’이 가능하다

문법 암기보다 실제 업무에서 안전하게 쓰는 판단 기준을 익힌다.

01

선택

SELECT 안의 계산은 함수,
업무 절차는 프로시저로 구
분

02

구현

주문관리 테이블과 연결해
조회·변경·배치를 직접 작성

03

검증

오류·실행계획·권한·
트랜잭션 경계까지 점검

하나의 주문관리 예제로 난이도를 단계적으로 높인다

PPT와 SQL 스크립트의 흐름이 같은 순서로 연결된다.

1

구분



함수와 프로시저의
역할

2

기초



문법·매개변수·반환

3

연동



조회·DML·배치

4

고급



예외·JSON·동적 SQL

5

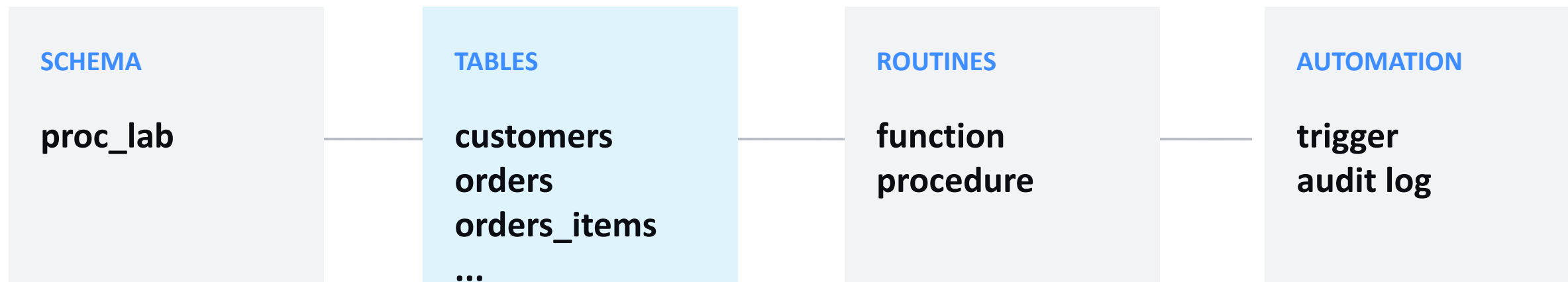
운영



성능·보안·테스트

루틴은 스키마 안에서 테이블과 함께 동작한다

함수와 프로시저는 독립 코드가 아니라 데이터 모델을 감싼 데이터베이스 객체다.



호출 결과는 테이블을 읽거나 바꾸고, 필요하면 감사·오류 로그로 남는다.

가장 큰 차이는 ‘반환값’보다 호출 위치와 트랜잭션 경계

PostgreSQL 함수는 읽기 전용으로 제한되지 않는다.

구분	FUNCTION	PROCEDURE
호출	SELECT fn_name(...)	CALL pr_name(...)
주 역할	값 계산·조회·변환	업무 절차·배치·변경
결과	스칼라·행·테이블 반환	OUT/INOUT 결과 가능
SQL 안 사용	WHERE·JOIN·SELECT에 가능	SQL 식 안에서는 불가
DML	가능 — VOLATILE이 기본	가능
트랜잭션 종료	COMMIT/ROLLBACK 불가	조건을 만족할 때 가능

일반 참고 표는 PostgreSQL 기준으로 두 곳을 보정한다

일반 비교표를 PostgreSQL의 실제 동작 기준으로 읽는다.

구분	Stored Procedure	함수 (Function, UDF)
역할	여러 SQL문을 묶어 한 번에 실행 (작업 절차)	계산해서 결과값을 돌려주는 함수
리턴값	있어도 되고 없어도 되고	반드시 1개 이상 결과 반환
호출 방법	CALL or EXEC	SELECT fn_함수() 형태
주 사용처	데이터 변경, 트랜잭션, 배치 작업	SELECT 안에서 계산, 데이터 변환
트랜잭션 제어	가능 (BEGIN/COMMIT/ROLLBACK)	불가능 (DML 금지)
부작용(DML)	INSERT, UPDATE, DELETE 가능	대부분 읽기 전용 (변경불가)
핵심 비유	작업 단위 (batch 작업 중심)	계산 도구 (연산기)

일반 비교 참고 이미지

① 함수도 DML 가능

권한이 있다면 INSERT·UPDATE·DELETE를 수행할 수 있다.

② 프로시저의 트랜잭션 제어는 조건부

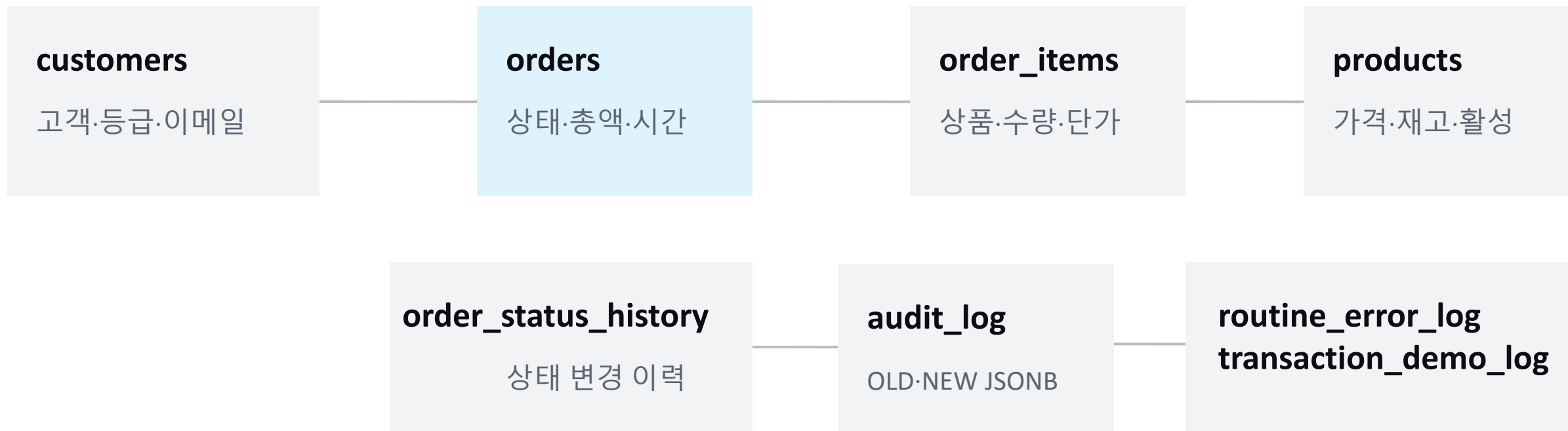
최상위 CALL이어야 하며 명시적 트랜잭션 · SECURITY DEFINER · SET 절 · EXCEPTION 블록 등의 제약을 확인해야 한다.

PostgreSQL

- 함수호출 키워드는 SELECT 함수명
- 프로시저 호출 키워드는 EXEC 대신 CALL 프로시저명.

실습 스키마는 주문 생성부터 감사 로그까지 이어진다

모든 루틴은 proc_lab 스키마 안의 동일한 테이블 집합을 사용한다.



SQL 실습파일 | CREATE SCHEMA → CREATE TABLE → 인덱스 → 샘플 데이터

루틴 실습은 '정의-호출-확인' 세 번으로 끝난다

CREATE만 성공했다고 완성된 것이 아니다.

01

정의

CREATE OR REPLACE
FUNCTION / PROCEDURE

02

호출

SELECT fn(...)
CALL pr(...)

03

확인

결과 행·테이블·로그
오류 메시지 점검

오류 테스트와 경계값 테스트를 의도적으로 실행해야 운영에서 안전하다.

PL/pgSQL 블록은 네 구역으로 읽으면 쉽다

DECLARE는 선택 사항이고 BEGIN과 END는 필수다.

- ① 매개 변수는 CREATE 문에서 선언
- ② 지역 변수는 DECLARE에 배치
- ③ 실제 업무 로직은 BEGIN 안에 작성
- ④ 오류 처리는 EXCEPTION에서 분기

```
CREATE FUNCTION fn_name(p_id bigint)
RETURNS numeric
LANGUAGE plpgsql
AS $function$
DECLARE
    v_total numeric := 0;
BEGIN
    SELECT sum(total_amount) INTO v_total
    FROM orders
    WHERE customer_id = p_id;
    RETURN coalesce(v_total, 0);
EXCEPTION
    WHEN OTHERS THEN RAISE;
END;
$function$;
```

구조: CREATE → DECLARE → BEGIN → EXCEPTION → END

한 문장으로 끝나면 LANGUAGE SQL이 더 단순하다

분기·변수·반복·예외가 필요해질 때 PL/pgSQL로 전환한다.

LANGUAGE SQL

- 한 개 또는 소수의 SQL 문
- 옵티마이저가 이해하기 쉬움
- 계산·단순 조회에 적합

```
SELECT round(p_amount * p_rate, 2);
```

LANGUAGE plpgsql

- DECLARE 지역 변수
- IF·CASE·LOOP 제어 흐름
- EXCEPTION·동적 EXECUTE

```
BEGIN ... IF ... RETURN ... END;
```

선택 기준: 복잡해 보이는 언어가 아니라 가장 단순한 표현을 사용한다.

첫 함수는 입력 하나를 결과 하나로 바꾼다

세액 계산은 테이블을 읽지 않는 순수 함수다.

RETURNS numeric으로 결과 형식을 선언

DEFAULT 0.10으로 생략 가능한 세율 제공

IMMUTABLE은 같은 입력에 같은 결과를 약속

STRICT는 NULL 입력이면 본문을 실행하지 않음

```
CREATE FUNCTION fn_tax_amount(  
    p_amount numeric,  
    p_tax_rate numeric DEFAULT 0.10  
)  
RETURNS numeric  
LANGUAGE sql IMMUTABLE STRICT  
AS $function$  
    SELECT round(p_amount * p_tax_rate, 2);  
$function$;
```

계산 함수도 입력 규칙을 스스로 지켜야 한다

잘못된 할인율을 조용히 계산하지 말고 명확한 오류로 중단한다.

IF로 단가·수량·할인율의 범위를 검사

RAISE EXCEPTION에 업무 메시지를 담음

ERRCODE를 지정하면 애플리케이션이 분류 가능

RETURN 전에 반올림 규칙을 한곳에 고정

```
IF p_quantity <= 0 THEN
    RAISE EXCEPTION '수량 오류: %', p_quantity
        USING ERRCODE = '22003';
END IF;
IF p_discount_percent
    NOT BETWEEN 0 AND 100 THEN
    RAISE EXCEPTION '할인율 오류: %',
        p_discount_percent;
END IF;
v_result := p_unit_price * p_quantity
    * (1 - p_discount_percent / 100.0);
RETURN round(v_result, 2);
```

STABLE 함수는 한 SQL 문 안에서 조회 결과를 재사용한다

고객별 누적 주문액을 SELECT의 계산 열처럼 붙일 수 있다.

orders 테이블을 조회하므로 IMMUTABLE
이 아님

취소 주문을 제외하는 업무 규칙을 함수
에 집중

SELECT 목록에서 고객마다 호출 가능

대량 행에서는 호출 횟수와 실행계획을
확인

```
CREATE FUNCTION fn_customer_order_total(  
    p_customer_id bigint)  
RETURNS numeric  
LANGUAGE sql STABLE STRICT  
AS $function$  
    SELECT coalesce(sum(o.total_amount), 0)  
    FROM orders AS o  
    WHERE o.customer_id = p_customer_id  
    AND o.order_status <> 'CANCELLED';  
$function$;
```

실습 SQL §05

여러 행은 RETURNS TABLE로 돌려준다

RETURN QUERY가 SELECT 결과 전체를 함수 결과로 연결한다.

반환할 열 이름과 타입을 RETURNS TABLE에 고정

RETURN QUERY가 SELECT 결과 전체를 누적

NULL 매개변수는 전체, 값이 있으면 상태 필터

호출 결과에 다시 WHERE·ORDER BY를 적용 가능

실습 SQL §06

```
CREATE FUNCTION fn_orders_by_status(  
    p_status text DEFAULT NULL)  
RETURNS TABLE(  
    order_id bigint, customer_name varchar,  
    order_status varchar, total_amount numeric)  
LANGUAGE plpgsql STABLE  
AS $function$  
BEGIN  
    RETURN QUERY  
    SELECT o.order_id, c.customer_name,  
           o.order_status, o.total_amount  
    FROM orders o  
    JOIN customers c USING (customer_id)  
    WHERE p_status IS NULL  
           OR o.order_status = p_status;  
END; $function$;
```

SELECT INTO는 조회 결과를 PL/pgSQL 변수에 담는다

STRICT를 붙이면 0행과 2행 이상을 예외로 구분할 수 있다.

%TYPE은 특정 열과 같은 타입

%ROWTYPE은 테이블 한 행 전체의 타입

FOUND는 직전 SQL이 행을 처리했는지 표시

STRICT는
NO_DATA_FOUND·TOO_MANY_ROWS 발생

실습 SQL §07

```
DECLARE
    v_order orders%ROWTYPE;
BEGIN
    SELECT o.*
        INTO STRICT v_order
        FROM orders AS o
        WHERE o.order_id = p_order_id;

    RETURN v_order;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION '주문 % 없음', p_order_id
            USING ERRCODE = 'P0002';
END;
```

함수도 DML이 가능하지만 호출의 부작용이 있을 수 있다

재고 차감 함수는 값을 반환하면서 products를 변경한다.

UPDATE ... RETURNING으로 변경과 결과를 한 번에 처리

재고 조건을 WHERE에 넣어 동시성 경쟁을 줄임

DML 함수는 VOLATILE이 기본이자 올바른 분류

SELECT가 데이터를 바꾼다는 사실을 이름과 문서로 명확히 표현

실습 SQL §08

```
UPDATE products
  SET stock_qty = stock_qty - p_quantity
  WHERE product_id = p_product_id
  AND active
  AND stock_qty >= p_quantity
  RETURNING stock_qty
  INTO v_remaining;

IF NOT FOUND THEN
  RAISE EXCEPTION
    '재고 부족: product_id=%',
    p_product_id;
END IF;

RETURN v_remaining;
```

프로시저 매개변수는 데이터의 이동 방향을 나타낸다

프로시저의 OUT/INOUT은 결과 행으로 표시되며 함수의 RETURNS와 역할이 다르다.

키워드	의미	실습에서의 사용
IN	입력 전용	함수·프로시저 기본값
OUT	출력 변수	CALL에서 NULL 자리 전달
INOUT	입력 후 수정해 출력	생성된 order_id 반환
DEFAULT	인수 생략 가능	뒤의 입력 인수도 기본값 필요
STRICT	NULL이면 본문 생략	함수 속성

프로시저는 여러 SQL을 하나의 업무 절차로 묶는다

주문 생성은 검증·재고 차감·주문·상세 입력을 순서대로 수행한다.

호출은 SELECT가 아니라 CALL

INOUT p_order_id에 새 주문번호를 반환

COMMIT을 내부에 넣지 않아 호출자가 원자성을 관리

중간 오류가 나면 같은 트랜잭션의 앞선 변경도 함께 취소 가능

```
CREATE PROCEDURE pr_create_order(  
    IN p_customer_id bigint,  
    IN p_product_id bigint,  
    IN p_quantity integer,  
    INOUT p_order_id bigint DEFAULT NULL)  
LANGUAGE plpgsql AS $procedure$  
BEGIN  
    PERFORM fn_reserve_stock(p_product_id, p_quantity);  
    INSERT INTO orders (...) VALUES (...)  
    RETURNING order_id INTO p_order_id;  
    INSERT INTO order_items (...) VALUES (...);  
END;  
$procedure$;  
CALL pr_create_order(1, 2, 2, NULL);
```

실습 SQL §09

OUT 매개변수는 처리 결과 메시지를 CALL 결과로 리턴

상태 변경이 없을 때도 오류 대신 의미 있는 메시지를 반환할 수 있다.

이전 상태를 SELECT ... FOR UPDATE로 잠금

GET DIAGNOSTICS ROW_COUNT로 실제 변경 행 확인

상태 변경 이력을 별도 테이블에 기록

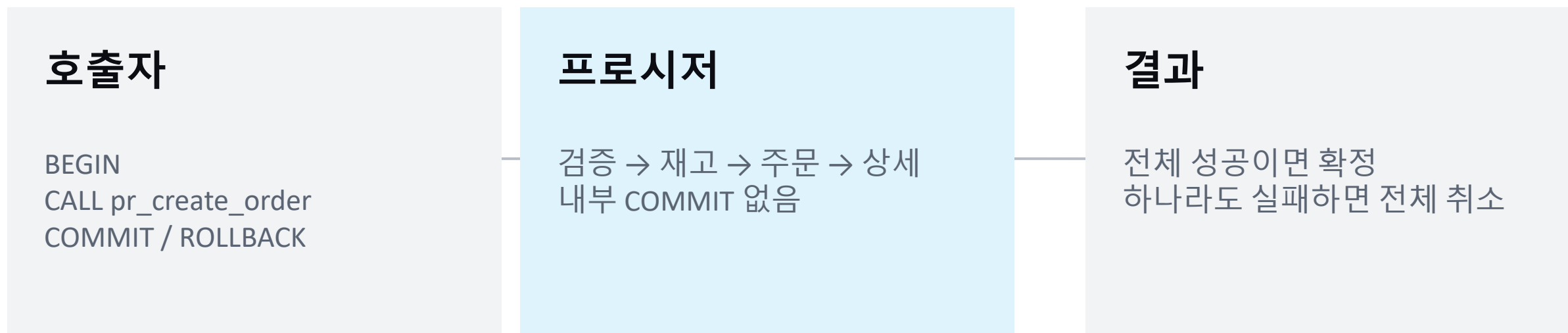
OUT 자리에는 SQL CALL에서 보통 NULL 전달

실습 SQL §10

```
CREATE PROCEDURE pr_change_order_status(  
  IN p_order_id bigint,  
  IN p_new_status text,  
  OUT p_message text)  
LANGUAGE plpgsql AS $procedure$  
BEGIN  
  UPDATE orders  
    SET order_status = p_new_status  
  WHERE order_id = p_order_id  
    AND order_status IS DISTINCT FROM p_new_status;  
  GET DIAGNOSTICS v_row_count = ROW_COUNT;  
  p_message := format('주문 %s 변경 완료', p_order_id);  
END;  
$procedure$;  
CALL pr_change_order_status(3, 'PAID', NULL);
```

기본 원칙은 호출자가 트랜잭션 전체를 소유하는 것이다

프로시저 내부 COMMIT은 꼭 필요한 장기 배치에서만 제한적으로 사용한다.



업무 원자성은 ‘프로시저 안의 문장 수’가 아니라 트랜잭션 경계로 결정된다.

COMMIT·ROLLBACK은 조건이 맞는 프로시저에서만 가능

DBeaver에서는 Auto-commit 상태에서 CALL 한 문장만 선택해 실행한다.

호출 환경	트랜잭션 종료	실무 판단
최상위 CALL / DO	가능	중간 SELECT 없이 중첩 CALL 가능
명시적 BEGIN 블록 안	불가	CALL 문을 단독 실행
함수 안	불가	함수는 트랜잭션 종료 금지
SECURITY DEFINER 프로시저	불가	권한 상승 루틴은 종료 금지
CREATE PROCEDURE의 SET 절	불가	구성값 복원 규칙 때문
EXCEPTION 블록 안	불가	내부 서브트랜잭션

UPSERT는 재실행해도 고객을 중복 생성하지 않는다

이메일을 소문자로 정규화한 생성 열에 UNIQUE 제약을 둔다.

email_normalized가 대소문자 차이를 제거

ON CONFLICT가 등록과 수정을 한 문장으로 처리

RETURNING customer_id로 신규·기존 ID를 동일하게 반환

재시도 가능한 API에 유리한 멍등성 패턴
즉 사용자가 실수로 버튼을 두 번 눌러도
주문·결제·재고 차감이 중복으로
처리되지 않도록 하는 것

실습 SQL §11

```
INSERT INTO customers(customer_name, email)
VALUES (btrim(p_customer_name), btrim(p_email))
ON CONFLICT (email_normalized)
DO UPDATE
  SET customer_name = EXCLUDED.customer_name,
      email = EXCLUDED.email,
      active = true
RETURNING customer_id INTO p_customer_id;

CALL pr_upsert_customer('한지민',
                        'JIMIN@example.com',
                        NULL);
```

행마다 다른 판단이 없다면 반복문보다 집합 SQL이 우선

루프는 제어가 필요할 때, 일괄 변경은 UPDATE 한 문장으로 처리한다.

FOR ... LOOP

사용할 때

- 행마다 외부 규칙이 다름
- 개별 NOTICE·예외 처리 필요
- 순차 의존성이 존재

대가: 반복 호출·락 유지·문맥 전환

SET-BASED UPDATE

우선 사용할 때

- 같은 계산을 여러 행에 적용
- 조건을 WHERE로 표현 가능
- ROW_COUNT만 필요

장점: 계획 최적화·적은 SQL 호출·간결성

실습 SQL §12는 LOOP, §13은 집합 UPDATE로 두 방식을 비교한다.

EXCEPTION 블록은 실패 범위와 오류 정보를 함께 제어

오류를 기록한 뒤 재발생시키면 같은 트랜잭션의 로그도 취소될 수 있다.

내부 BEGIN ... EXCEPTION은 서브트랜잭션처럼 동작

GET STACKED DIAGNOSTICS로 SQLSTATE·상세·문맥 수집

로그를 보존하려면 예외를 흡수하거나 별도 로깅 경로 설계

WHEN OTHERS 남용보다 예상 가능한 오류를 먼저 분기

실습 SQL §14

```
BEGIN
  UPDATE products ...; -- 재고 복구
  UPDATE orders ...;  -- 주문 취소
EXCEPTION
  WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS
      v_state = RETURNED_SQLSTATE,
      v_message = MESSAGE_TEXT,
      v_detail = PG_EXCEPTION_DETAIL,
      v_context = PG_EXCEPTION_CONTEXT;
    INSERT INTO routine_error_log(...)
      VALUES ('pr_cancel_order', v_state,
              v_message, v_detail, v_context);
    p_message := format('취소 실패 [%s] %s',
                        v_state, v_message);
END;
```

JSONB 함수는 주문 데이터를 API 구조로 조립한다

주문.고객.상품 상세를 한 번의 함수 호출로 영수증 형태로 반환한다.

jsonb_build_object로 키-값 객체 구성

jsonb_agg로 여러 주문상품을 배열화

항목이 없을 때 []::jsonb로 일관된 결과

대형 JSON은 네트워크.메모리 비용도 함께 점검

```
SELECT jsonb_build_object(
  'order_id', o.order_id,
  'customer', jsonb_build_object(...),
  'items', coalesce((
    SELECT jsonb_agg(
      jsonb_build_object(...)
    )
  FROM order_items oi
  JOIN products p USING (product_id)
  WHERE oi.order_id = o.order_id
), '[]'::jsonb)
)
FROM orders o
JOIN customers c USING (customer_id);
```

동적 SQL은 식별자와 값을 분리해야 안전하다

테이블·컬럼은 format, 값은 USING으로 전달한다.

테이블은 regclass로 실제 객체를 먼저 해석

컬럼은 format('%I', name)으로 안전하게 인용

값을 문자열 결합하지 않고 \$1 + USING으로 바인딩

범용 ::text 비교는 인덱스를 가릴 수 있어 대용량에는 정적 SQL 우선

실습 SQL §16

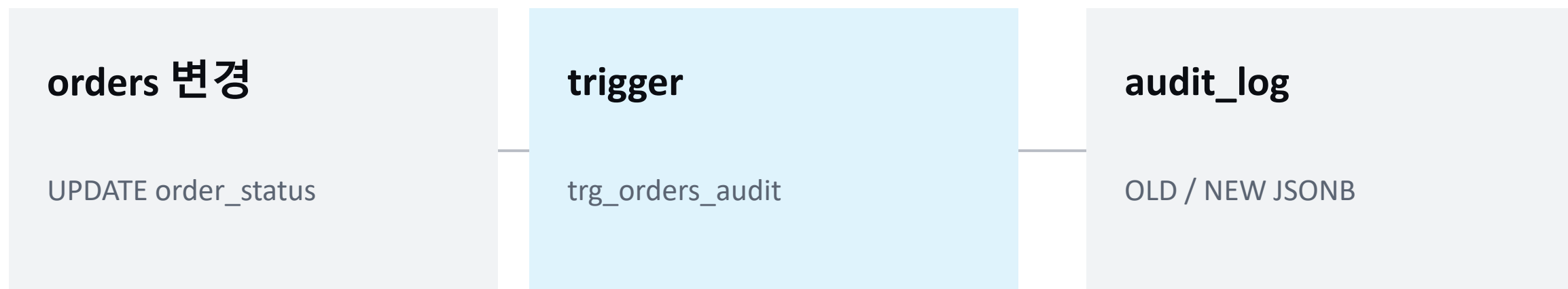
```
v_sql := format(
    'SELECT count(*)
      FROM %s
     WHERE %I::text = $1',
    p_table, -- regclass
    p_column -- name
);

EXECUTE v_sql
  INTO v_count
  USING p_value;

RETURN v_count;
```

트리거는 '함수'를 이벤트에 연결해 자동 실행한다

주문 INSERT·UPDATE·DELETE가 발생하면 OLD/NEW를 JSONB 감사 로그로 남긴다.



--트리거 함수에는 일반 매개변수가 없어도 PostgreSQL이 다음과 같은 특별한 값을 자동으로 제공합니다.
`RETURNS trigger | TG_TABLE_NAME | TG_TABLE_SCHEMA | OLD | NEW | TG_OP`

AFTER 트리거는 원본 작업이 성공한 뒤 로그를 기록하고, 같은 트랜잭션에서 함께 확정된다.

실습 SQL §17

변동성 표시는 옵티마이저에게 하는 약속이다

잘못 지정하면 빠른 것이 아니라 잘못된 결과를 만들 수 있다.

IMMUTABLE

같은 입력 → 항상 같은 결과

순수 계산·함수 인덱스
fn_tax_amount

STABLE

한 SQL 문 안에서 안정적

테이블 조회 함수
fn_customer_order_total

VOLATILE

호출마다 달라질 수 있음

DML·random·clock_timestamp
fn_reserve_stock

기본값은 VOLATILE이다. 확실할 때만 더 강한 약속으로 바꾼다.

실습 SQL §18

성능은 PL/pgSQL 문법보다 내부 SQL에서 결정된다

루틴을 블랙박스로 두지 말고 핵심 SQL의 실행계획을 확인한다.

- | | | |
|----|----------------|--------------------------------------|
| 01 | 집합 SQL 우선 | 루프보다 한 번의 UPDATE·INSERT SELECT |
| 02 | 인덱스 보존 | WHERE 컬럼을 불필요한 함수로 감싸지 않기 |
| 03 | 변동성 정확히 | IMMUTABLE·STABLE·VOLATILE을 실제 동작과 일치 |
| 04 | 동적 SQL 최소화 | 매번 다시 계획되는 EXECUTE는 필요한 곳에만 |
| 05 | 밖으로 꺼내 EXPLAIN | 함수 내부 핵심 SQL을 단독으로 ANALYZE |

SECURITY DEFINER는 권한과 search_path를 함께 잠가야 한다

필요한 최소 권한만 위임하고 PUBLIC 실행 권한을 검토한다.

SECURITY INVOKER

기본값

호출자의 권한으로 실행

일반 조회·업무 함수에 우선

SECURITY DEFINER

소유자의 권한으로 실행

권한 상승이 목적일 때만

객체명을 스키마로 한정

안전 체크

```
SET search_path =  
pg_catalog, proc_lab, pg_temp
```

```
REVOKE ... FROM PUBLIC  
GRANT EXECUTE ...
```

SECURITY DEFINER 프로시저는 COMMIT·ROLLBACK을 실행할 수 없다.

실습 SQL §19

pg_proc 카탈로그는 루틴 설계를 한눈에 점검하게 해준다

종류.인수.언어.변동성.병렬 안전성.보안 속성을 inventory 뷰로 만든다.

prokind: 함수 f / 프로시저 p

provolatile: immutable i / stable s / volatile v

proparallel: safe s / restricted r / unsafe u

prosecdef: SECURITY DEFINER 여부

```
SELECT n.nspname AS routine_schema,  
       p.proname AS routine_name,  
       p.prokind,  
       pg_get_function_identity_arguments(  
         p.oid  
       ) AS arguments,  
       p.provolatile,  
       p.proparallel,  
       p.prosecdef  
FROM pg_proc p  
JOIN pg_namespace n  
  ON n.oid = p.pronamespace  
WHERE n.nspname = 'proc_lab';
```

실습 SQL §21

루틴 테스트는 결과뿐 아니라 ‘부작용’도 확인한다

반환값, 변경된 테이블, 로그, 락과 권한을 함께 검증한다.

테스트	입력 예	확인 항목
정상	대표 입력	예상 결과·변경 행 확인
경계	0·NULL·최솟값·최댓값	STRICT·CHECK·범위 오류
실패	없는 ID·재고 부족	SQLSTATE·오류 메시지
반복	같은 요청 두 번	UPSERT·멥등성
동시성	같은 재고/주문 경쟁	락·조건부 UPDATE
권한	일반 역할 호출	GRANT·SECURITY DEFINER

자주 만나는 오류는 호출 문맥과 타입에서 시작한다

메시지를 숨기지 말고 원인-수정 패턴으로 읽는다.

오류	주 원인	수정 방향
procedure ... does not exist	NULL 타입/인수 순서 불일치	명시적 캐스팅·시그니처 확인
invalid transaction termination	CALL이 트랜잭션 블록 안	Auto-commit에서 단독 CALL
query returned more than one row	INTO STRICT에 여러 행	조건 강화·LIMIT 검토
column reference is ambiguous	매개변수와 열 이름 충돌	p_ 접두어·테이블 별칭
permission denied	호출/테이블 권한 부족	GRANT 또는 안전한 DEFINER
function result type mismatch	RETURNS TABLE 타입 불일치	명시적 캐스팅·열 순서 확인

루틴을 만들기 전에 다섯 질문으로 결정한다

재사용할 가치가 있는 업무 규칙만 데이터베이스 객체로 만든다.

질문	우선 선택
SELECT 안에서 값이 필요한가?	FUNCTION
여러 SQL을 한 업무 절차로 묶는가?	PROCEDURE
테이블 이벤트에 자동 반응하는가?	TRIGGER FUNCTION
단순 조회를 재사용하려는가?	VIEW도 먼저 검토
한 번의 SQL로 충분한가?	루틴을 만들지 않아도 됨

종합평가

함수·프로시저·트리거

proc_lab 주문관리 데이터로 8개 객체를 정의하고
호출·검증 SQL까지 제출합니다.

8문제는 계산에서 자동화까지 단계적으로 확장됩니다

FUNCTION · 35점

PROCEDURE · 45점

TRIGGER · 20점

문제 1-3

단일값 · 행 집합 · JSONB
반환 형태와 변동성 선택

문제 4-6

주문 생성 · 상태 전환 · 맥등 할인
잠금과 DML 흐름

문제 7-8

감사 로그 · 총액 동기화
OLD · NEW · TG_OP 활용

문제 1 · 고객 등급에 따른 할인율을 함수로 구현하세요

정의 및 검색시 사용

fn_grade_discount_rate(p_grade text)

customer_id	customer_name	customer_grade	고객 등급별 할인율
1	김민준	GOLD	0.10
2	이서연	VIP	0.15
3	박지훈	SILVER	0.05
4	최유리	BASIC	0
5	정도윤	GOLD	0.10

필수 조건

- 앞뒤 공백과 대소문자 무시
- 그 외 값은 0 반환
- LANGUAGE sql
- IMMUTABLE · STRICT · PARALLEL SAFE
- customers와 함께 호출해 결과 확인

문제 2 · 재고 미달 상품 조회 RETURNS TABLE로 행 집합을 반환하세요

테이블 반환 구조

`fn_products_below_stock(p_stock_qty
integer)`

RETURNS TABLE

- 여러 행과 여러 컬럼을 반환한다.
- 호출 결과를 일반 테이블처럼 FROM 절에서 사용할 수 있다.

검증 기준

- active = true
 - stock_qty < p_stock_qty
 - 재고 오름차순, 상품번호 오름차순
- 호출값 70의 예상 상품
모니터(40) · 웹캠(60)

문제 3 · 고객 주문 요약을 JSONB 한 객체로 만드세요

JSON 구조

```
fn_customer_order_summary_json(  
    p_customer_id bigint)  
{  
    "customer": {  
        "name": "김민준",  
        "email": "minjun@example.com",  
        "grade": "GOLD",  
        "customer_id": 1  
    },  
    "order_count": 2,  
    "total_amount": 305000.00  
}
```

업무 규칙

- CANCELLED 주문 제외
- 주문이 없으면 count와 합계는 0
- 고객이 없으면 SQLSTATE P0002
- LANGUAGE plpgsql · STABLE · STRICT
- 고객 1 예상: 2건 · 305000

문제 4 · 주문 생성 흐름을 한 프로시저로 묶으세요

입력과 잠금

```
proc_lab.pr_create_order(  
  IN      p_customer_id bigint,  
  IN      p_product_id  bigint,  
  IN      p_quantity integer,  
  INOUT   p_order_id  bigint DEFAULT NULL  
)
```

수량 · 고객 활성화 · 상품 활성화 검증

상품 행 FOR UPDATE

재고 부족 여부 확인

생성과 반환

재고 차감 → PENDING 주문 생성

→ order_items 생성

$\text{total_amount} = \text{unit_price} \times \text{quantity}$

RETURNING으로 주문번호 반환

CALL 입력 예: 고객 1 · 상품 2 · 수량 3

문제 5 · 상태 전환은 허용된 방향으로만 이동해야 합니다

허용 전환

```
proc_lab.pr_change_order_status(  
    IN p_order_id bigint,  
    IN p_new_status text,  
    OUT p_message text  
)
```

PENDING → PAID | CANCELLED

PAID → SHIPPING | CANCELLED

SHIPPING → COMPLETED

같은 상태 요청은 변경하지 않고 안내 메시지 반환

구현 조건

- 주문 행 FOR UPDATE
- 미존재 주문은 P0002
- 허용되지 않은 전환은 P0001
- orders 갱신 + history 기록
- OUT p_message 반환
- 주문 2를 COMPLETED로 변경해 확인

문제 6 · 같은 할인 배치는 한 번만 실행되게 만드세요

먹등성 보장

```
proc_lab.pr_bulk_discount_once(  
  IN p_batch_key text,  
  IN p_discount_percent numeric,  
  IN p_min_price numeric,  
  OUT p_affected integer  
)
```

배치키 · 할인율(0 초과 50 이하)

최소가격(0 이상) 검증

bulk_discount_runs 선점

조건 상품 할인 → 완료 상태와 건수 저장

재요청

동일 키 + 동일 조건

→ 가격 재할인 금지

→ 최초 affected 그대로 반환

동일 키 + 다른 조건 → 예외

ON CONFLICT와 ROW_COUNT 활용

같은 CALL을 두 번 실행해 검증

문제 7. 주문 변경 이력을 자동 감사 로그로 남기세요

트리거 설계

fn_audit_orders()
RETURNS trigger

```
CREATE TRIGGER trg_orders_audit  
AFTER INSERT OR UPDATE OR DELETE  
ON proc_lab.orders  
FOR EACH ROW  
EXECUTE FUNCTION proc_lab.fn_audit_orders();
```

감사 · 보안

- table_name · TG_OP · order_id 저장
- OLD/NEW 전체 행을 JSONB로 저장
- changed_by = session_user
- SECURITY DEFINER
- search_path 고정
- note 변경 후 audit_log 확인

문제 8 · 주문 상세가 바뀌면 총액도 자동으로 맞추세요

자동 재계산

트리거 함수명

proc_lab.fn_sync_order_total()

트리거명

trg_order_items_sync_total

sum(line_amount), 없으면 0

orders.total_amount와 updated_at 갱신

검증 시나리오

주문 3: 118000

상품 1을 수량 2로 추가 → 296000

수량 3으로 변경 → 385000

삭제 → 118000

UPDATE에서 order_id가 바뀔 경우

OLD 주문과 NEW 주문을 모두 재계산

제출 파일은 실행 결과까지 스스로 증명해야 합니다

객체 정의

함수 3개
프로시저 3개
트리거 함수·트리거 2세트
스키마명 명시

호출·검증

SELECT·CALL 포함
테스트 DML 포함
메시지·행·감사로그 확인

데이터 보존

수동커밋 상태 - TCL
BEGIN / ROLLBACK 권장
반복 실행 가능하게 작성
위에서 아래로 오류 없이 실행

제출 전 확인

[최종 제출 내용]

- 각 문제 별 함수,프로시저,트리거 등을 정의하고
결과확인합니다
- 주어진 문제의 각문제 하단의 빈 공간에 작성하세요

[최종 제출 파일]

- day04_쿼리_홍길동.sql

평가 기준

FUNCTION · 35점

PROCEDURE · 45점

TRIGGER · 20점

슬랙>다이렉트 메시지

•수고하셨습니다.

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.